

Procesadores Segmentados

Arquitectura de computadores - Semana 2, 3 y 4

Grado en *Ingeniería Informática*

Departamento de Informática. Universidad de Jaén



Objetivos

General

Conocer los aspectos fundamentales de los procesadores segmentados

Específicos

- Comprender los fundamentos de la arquitectura del conjunto de instrucciones RISC-V.
- Estudiar el concepto de segmentación en el procesamiento de instrucciones RISC-V.
- Comprender los riesgos estructurales por la limitación de recursos físicos del hardware.
- Identificar los riesgos de datos debido a dependencias entre instrucciones.
- Comprender los riesgos de control por el desvío del flujo de instrucciones
- Examinar las técnicas de optimización del reordenamiento de código y el desenrollado de bucles.
- Entender las técnicas de predicción de saltos.

Nivel de Abstracción



Niveles de representación/interpretación

Lenguaje de Programación de alto nivel. (Ej. C)

```
temp = v[k];  
v[k] = v[k+1];  
v[k+1] = temp;
```

Compilador

Programa en lenguaje ensamblador (Ej., RISC-V)

```
lw    x3, 0(x10)  
lw    x4, 4(x10)  
sw    x4, 0(x10)  
sw    x3, 4(x10)
```

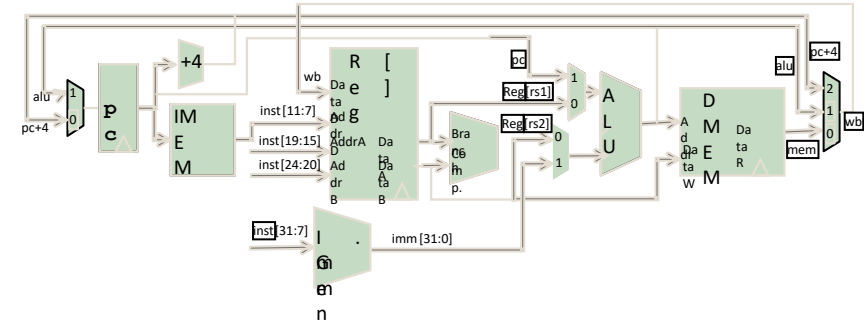
Ensamblador

Programa en lenguaje máquina (RISC-V)

```
1000 1101 1110 0010 0000 0000 0000 0000  
1000 1110 0001 0000 0000 0000 0000 0100  
1010 1110 0001 0010 0000 0000 0000 0000  
1010 1101 1110 0010 0000 0000 0000 0100
```

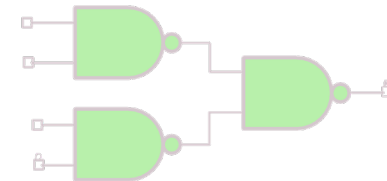
Implementación de la arquitectura

Descripción Arquitectura hardware
(Ej. Diagrama de bloques)



Electrónica digital

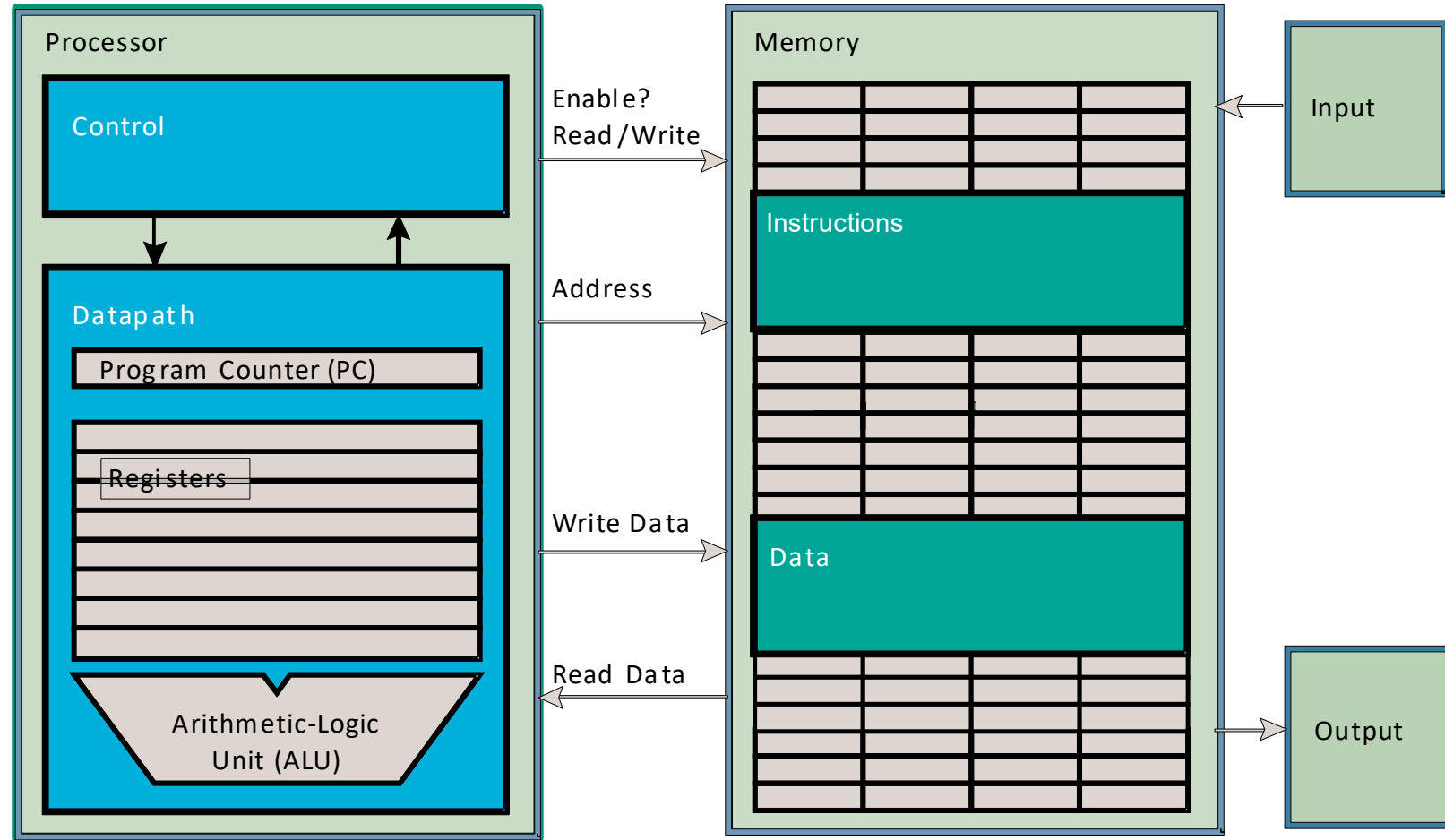
Descripción de circuitos lógicos
(Diagramas esquemáticos de circuitos)



Niveles de representación/interpretación

Conceptos iniciales

- **Procesador** (CPU) es la parte activa del ordenador que realiza todo el trabajo (manipulación de datos y toma de decisiones).
- **Datapath** (o ruta de datos) es la parte del procesador que contiene el hardware necesario para realizar las operaciones requeridas por el procesador.
- **Control** es la parte del procesador (también en hardware) que indica al *datapath* lo que debe hacer.



Rendimiento - Ejemplo ilustrativo



	Deportivo	Bus
Capacidad de pasajeros	2	50
Velocidad	200 km/h	50 km/h
Kilómetros por litro	5 km/l	2 km/l

**Suponemos un viaje de 50 km
(suponiendo que debe de regresar instantáneamente)**

	Deportivo	Bus
Tiempo de viaje	15 min	60 min
Tiempo por 100 pasajeros	750 min (50 viajes; 2 personas por viaje)	120 min (2 viajes; 50 personas por viaje)
Litros por pasajero	5 litros	0.5 litros

Analogía con el computador

Vehículo	Computador
Tiempo de viaje	Tiempo de ejecución del programa Ej.: Tiempo para actualizar la pantalla
Tiempo por 100 pasajeros	Rendimiento: Ej.: Número de solicitudes al servidor manejadas por hora
Kilómetros por litro	Energía por tarea*. Ej.: cuántas películas puede ver por carga de batería o factura energética del centro de datos.

* Nota: La energía consumida no es una buena medida, ya que una CPU de baja potencia puede funcionar durante mucho tiempo para completar una tarea consumiendo más energía que un computador más rápido que funcione a mayor potencia durante menos tiempo.

Tiempo de Programa

El número global de **instrucciones por programa (N)** está determinado por:

- Tarea
- Algoritmo, por ejemplo, $O(N^2)$ frente a $O(N)$
- Lenguaje de programación
- Compilador
- Arquitectura del conjunto de instrucciones (ISA)

$$T = N \times CPI \times T_c$$

Tiempo de Programa

Los Ciclos de reloj por instrucción Medios (**CPI**) están determinado por:

- ISA
- Implementación del procesador (o microarquitectura)
 - Diseño RISC-V de ciclo único, $CPI = 1$
 - Instrucciones complejas (P. Ej. strcpy), $CPI \gg 1$
 - Procesadores superescalares, $CPI < 1$ (los veremos más adelante)

$$T = N \times CPI \times Tc$$

Tiempo de Programa

El **tiempo de ciclo** ($T_c = 1/\text{Frecuencia}$) está determinado por:

- Microarquitectura del procesador
 - Determina la ruta crítica a través de las puertas lógicas
- Tecnología
 - P. Ej. 5nm frente a 28nm
- Presupuesto de energía
 - P. Ej. Los voltajes más bajos reducen la velocidad de los transistores

$$T = N \times CPI \times T_c$$

Tiempo de Programa

Ejemplo de ejecución de una tarea en dos procesadores:

	Procesador A	Procesador B
Número de instrucciones (N)	1 Millón	1.5 Millón
Ciclos por instrucción (CPI) medios	2.5	1
Frecuencia de reloj (1/T _c)	2.5 GHz	2 GHz
Tiempo de ejecución (T)	1 ms	0.75 ms

$$T = N \times CPI \times T_c$$

El procesador B es más rápido para esta tarea, a pesar de ejecutar más instrucciones y tener una velocidad de reloj más lenta.

Introducción a la segmentación (Pipeline)



Segmentación

La segmentación (pipelining)

Es una técnica de implementación por la cual se solapa el procesamiento de múltiples instrucciones y es una de las claves para hacer CPUs rápidas.

En RISC-V

- Se busca una nueva instrucción en **cada ciclo** de reloj.
- El objetivo es conseguir **etapas lo más equilibradas** posibles.
- El tiempo de ejecución de cada instrucción **será similar**, ya que todas las etapas deben de tener la misma duración.
- Todas las instrucciones pasarán por **todas las etapas**.
 - En un procesador secuencial, cada instrucción pasaba únicamente por las etapas necesarias.

Segmentación – Ejecución secuencial

Fases de las instrucciones.

Instr	IF = 200ps	ID = 100ps	ALU = 200ps	MEM=200ps	WB = 100ps	Total
add	X	X	X		X	600ps
beq	X	X	X			500ps
jal	X	X	X			500ps
lw	X	X	X	X	X	800ps
sw	X	X	X	X		700ps

Máxima frecuencia de reloj: $f_{\max} = 1/800\text{ps} = 1.25 \text{ GHz}$

¿Cómo aumentar el rendimiento?

Segmentación

La segmentación (pipelining)

Número de Instrucción	Ciclo de reloj								
	1	2	3	4	5	6	7	8	9
I	IF	ID	EX	MEM	WB				
I+1		IF	ID	EX	MEM	WB			
I+2			IF	ID	EX	MEM	WB		
I+3				IF	ID	EX	MEM	WB	
I+4					IF	ID	EX	MEM	WB

Segmentación

Rendimiento en un procesador segmentado

- La segmentación incrementa el rendimiento aumentando la productividad, en lugar de reducir el tiempo de ejecución de cada instrucción individual.
 - Es más, la segmentación incrementa el tiempo de ejecución de una instrucción individual.
- Los programas ejecutan millones de instrucciones por segundo, por lo que la productividad es una medida importante.
- En una situación ideal, siendo ***K*** el núm. de etapas, tenemos que:

$$T_{Segmentado} = \frac{T_{SinSegmentar}}{K}$$

- Factores que limitan la frecuencia de instrucciones son el retardo de los cerrojos (las entradas a la ALU), saltos, sesgo del reloj, etc.

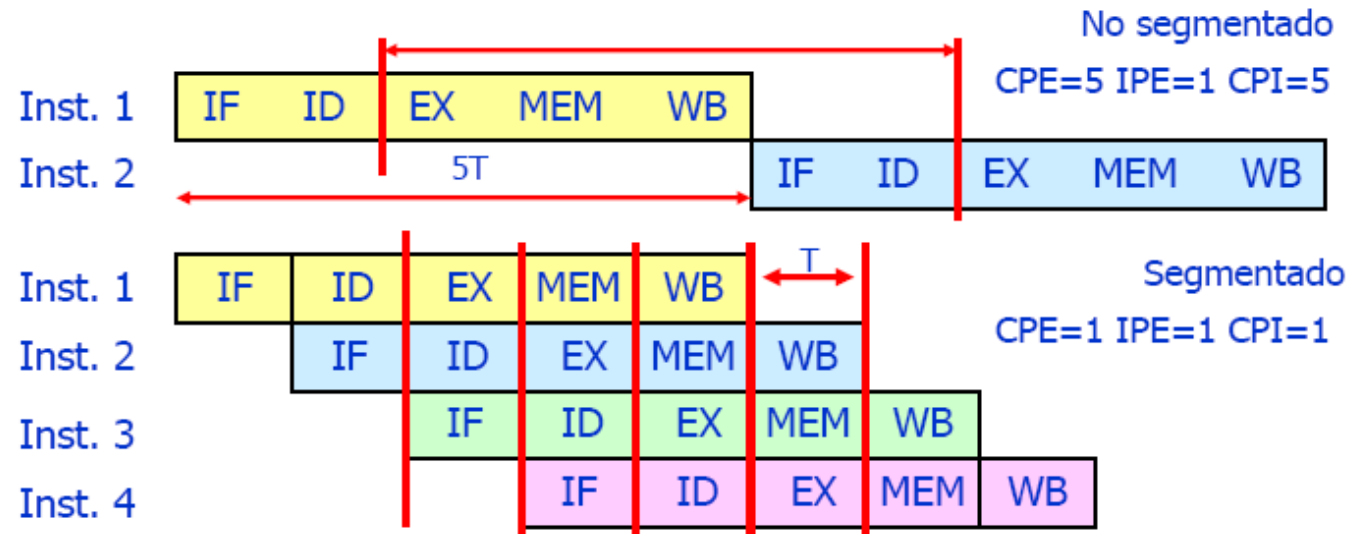
Segmentación

Tiempo de ejecución

- El tiempo de ejecución T_{CPU} , en concreto el **CPI**, puede reescribirse teniendo en cuenta los elementos de la arquitectura donde **CPE** es el número de ciclos entre la emisión de una instrucción y la emisión de la siguiente instrucción e **IPE** es el número de instrucciones que pueden emitirse.
- La emisión de instrucciones se refiere al proceso en el cual una unidad de procesamiento (como una CPU) toma instrucciones de un flujo de instrucciones y las coloca en su etapa inicial de ejecución en el pipeline (paso de la etapa ID a EX).

$$CPI = \frac{CPE}{IPE}$$

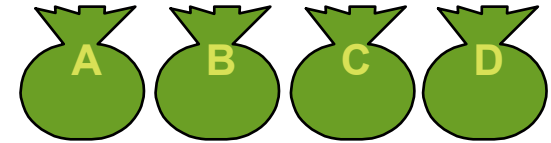
$$T_{CPU} = N \times \frac{CPE}{IPE} \times T_C$$



Ejemplo ilustrativo

Domingos de Colada

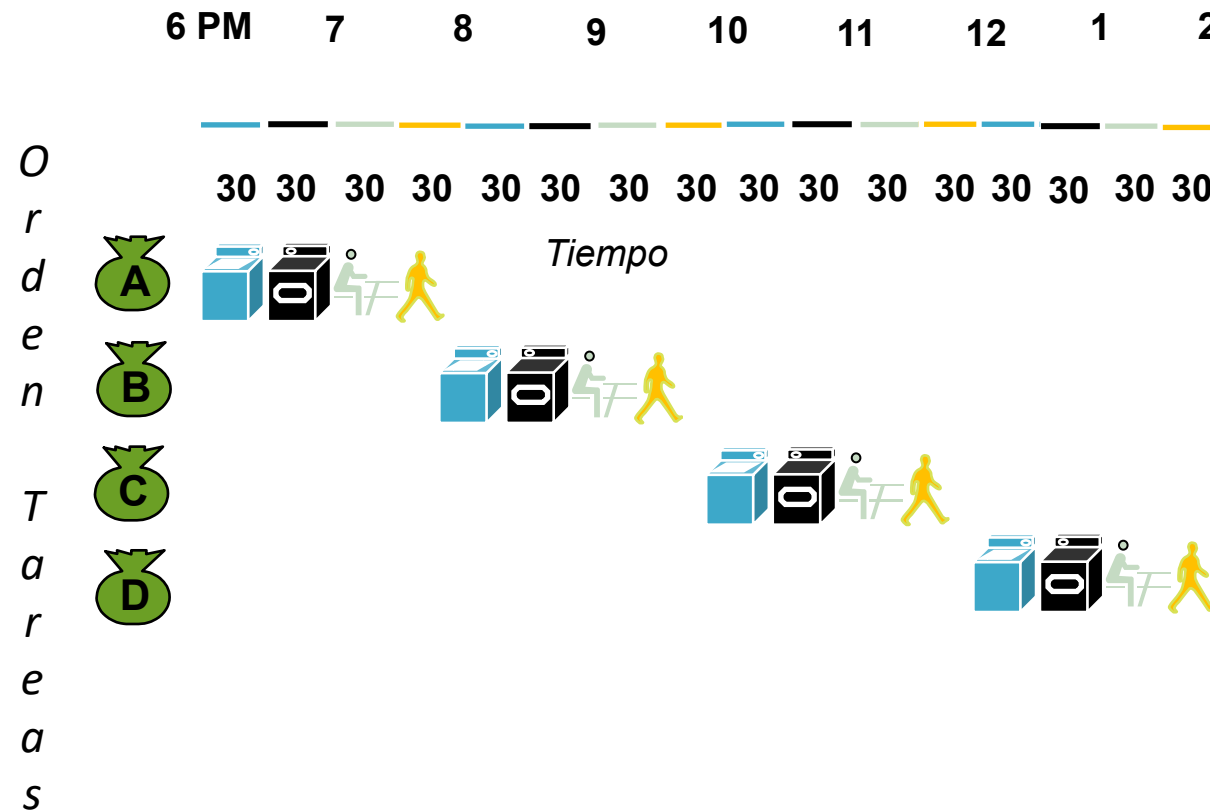
- Ana, Bernardo, Carlos y Daniela viven juntos en el mismo piso, y los domingos cada uno tiene preparada una cesta de ropa sucia para lavar.
- Cada cesta de ropa implica 4 tareas:
 - Lavar
 - Secar
 - Doblar
 - Colocar en los cajones
- Cada tarea lleva aproximadamente 30 minutos.



Ejemplo ilustrativo

Colada secuencial

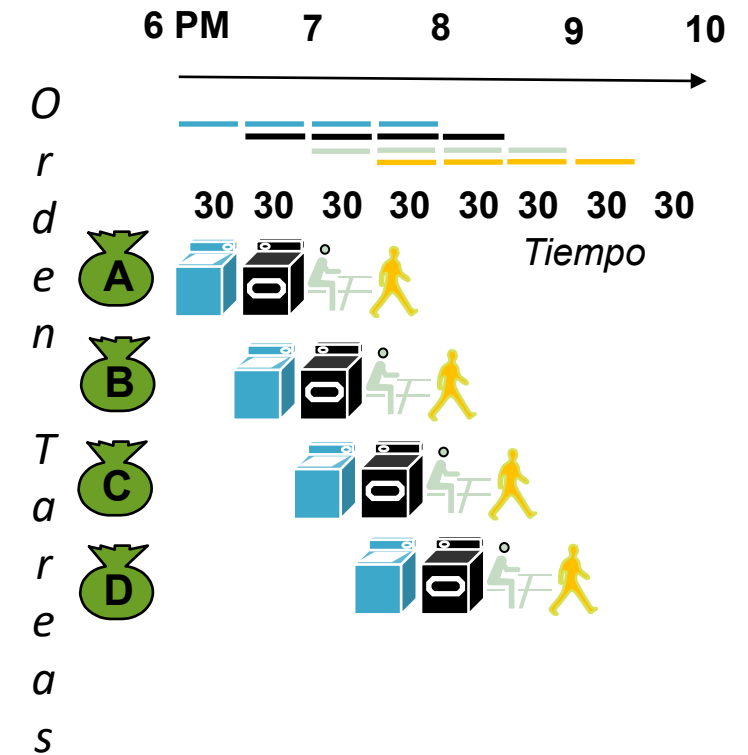
- La colada secuencial tarda **8 horas** para las 4 cestas
- Primero comienza Ana, cuando ella termina, continúa Bernardo. Cuando Bernardo termina, comienza Carlos y, finalmente, cuando él termina, comienza Daniela.



Ejemplo ilustrativo

Colada de cauce (pipeline)

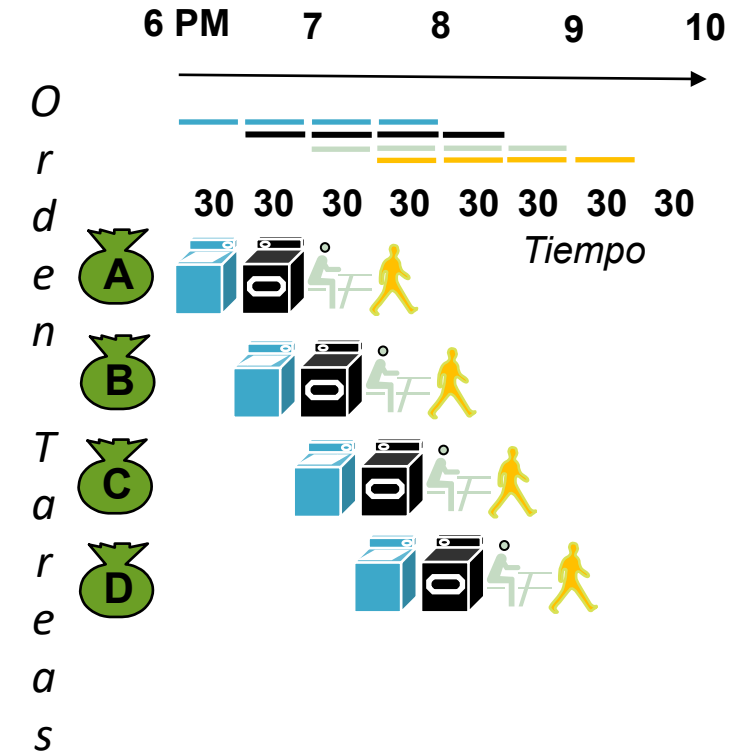
- La colada en cauce tarda **3,5 horas** para 4 cestas.



Ejemplo ilustrativo

Consideraciones

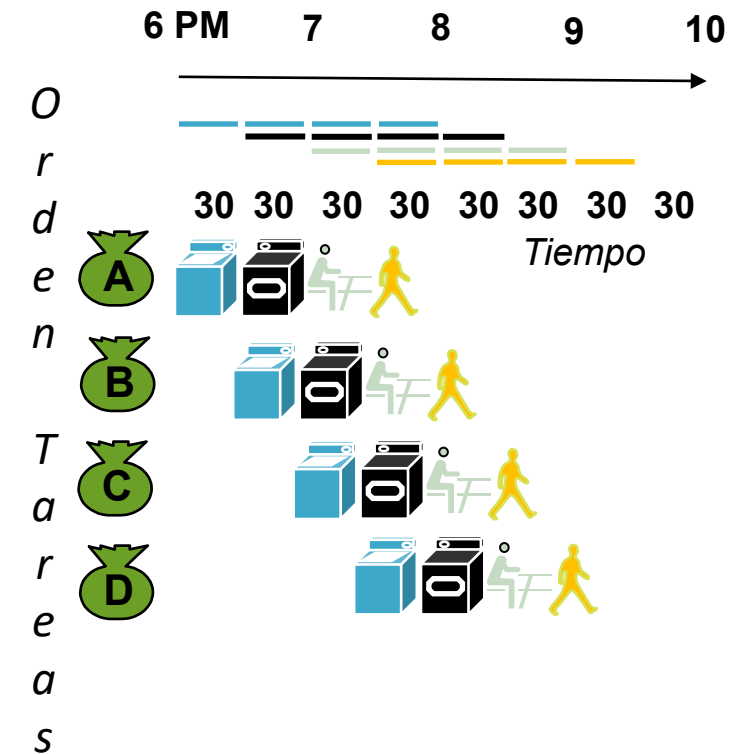
- El *pipelining* (segmentación) no ayuda a reducir la latencia (tiempo) de una sola tarea, sino al rendimiento de toda la carga de trabajo.
 - Múltiples tareas operan simultáneamente utilizando diferentes recursos.
 - Rendimiento potencial (ideal) = Número de etapas del cauce
- $$T_{Segmentado} = \frac{T_{SinSegmentar}}{K}$$
- El tiempo necesario para "llenar" el cauce y "vaciarlo" reduce la velocidad:
 - 2,3X frente a 4X en este ejemplo.



Ejemplo ilustrativo

Suposiciones de mejora

- Una nueva lavadora **reduce el tiempo** de lavado a **20 minutos**
- Una nueva secadora **reduce el tiempo** de secado a **20 minutos**
- ¿Cuánto más rápido es el cauce?
 - La velocidad del cauce está limitada por su tarea (etapa) más lenta.
 - Las duraciones desequilibradas entre tareas (etapas) del cauce no reducen la velocidad.



Pipeline - Encauzamiento

Conclusiones

- El tiempo en procesar un programa depende de la complejidad de sus instrucciones, la arquitectura y la tecnología
- El *pipelining* (encauzamiento) aumenta la frecuencia de reloj y las "instrucciones por segundo".
 - Sin embargo, reduce el tiempo necesario para completar las instrucciones de manera global.
- Medidas de rendimiento
 - Diferentes medidas en función del objetivo
 - Tiempo de respuesta
 - Tareas por segundo
 - Energía por tarea

Recursos

Descripción

Para ampliar tus conocimientos sobre los contenidos de esta semana te recomendamos que consultes los recursos indicados a continuación.

Recursos

- Arquitectura de Computadores de Julio Ortega, Alberto Prieto y Mancia Anguita. Ediciones Paraninfo – 978849732274
- Great Ideas in Computer Architecture (Machine Structures) CS 61C at UC Berkeley with Dan Garcia, Justin Yokota - Spring 2023, Lección 21, 22 y 23([link](#))
- *Nota: Se reconoce los derechos de parte del material a Dan Garcia y a Justin Yokota en CS 61C at UC Berkeley with Spring 2023*